

دليل Git & GitHub الشامل

من المبتدئ إلى المحترف — خطوة بخطوة عبر الطرفية

دليل عملي تطبيقي 🏠

البدء مع Git — التثبيت والإعداد الأولي

قبل أي شيء، تأكد من تثبيت Git على جهازك وإعداد هويتك عبر الطرفية.

تثبيت Git 📦

⚙️ الإعداد الأولي (مرة واحدة فقط)

بعد التثبيت، عرّف نفسك لـ Git عبر الطرفية:

Linux (Ubuntu/Debian)

```
sudo apt update
sudo apt install git
```

macOS

```
brew install git
```

Windows

حمل المثبت من **git-scm.com** ثم شغل Git Bash

```
# تعيين اسمك
git config --global user.name "اسمك"

# تعيين بريدك الإلكتروني
git config --global user.email "email@example.com"

# تعيين المحرر الافتراضي (اختياري)
git config --global core.editor "code --wait"

# التحقق من الإعدادات
git config --list

# التحقق من الإصدار
git --version
```

✅ بمجرد تنفيذ هذه الأوامر، Git جاهز للاستخدام على جميع مشاريعك. ✅

إنشاء المشاريع والتعامل معها في Git

التعامل مع الفروع (Branches) 🌿

إنشاء مستودع جديد 🚀

```
# أنشئ مجلد المشروع وادخل إليه
mkdir my-project
cd my-project

# داخل المجلد Git هَبَّعْ
git init

# أنشئ ملفاً وأضفه للمرحلة
touch README.md
git add README.md

# أو أضف كل الملفات دفعة واحدة
git add .

# سجِّل أول commit
git commit -m "إضافة ملف README"
```

دورة العمل اليومية ↻

```
# تحقق من حالة الملفات
git status

# اعرض التغييرات قبل الإضافة
git diff

# أضف ملفات محددة
git add filename.txt

# سجِّل التغييرات مع رسالة واضحة
git commit -m "وصف التغيير بوضوح"

# اعرض commits سجل إل
git log --oneline
```

```
# اعرض الفروع الموجودة
git branch

# أنشئ فرعاً جديداً
git branch feature/login

# انتقل إلى الفرع الجديد
git checkout feature/login

# أو أنشئ وانتقل في أمر واحد
git checkout -b feature/login

# ادمج فرعاً في الفرع الحالي
git checkout main
git merge feature/login

# احذف فرعاً بعد دمج
git branch -d feature/login
```

💡 نصيحة: استخدم أسماء وصفية للفروع مثل `feature/اسم-الميزة` أو `fix/اسم-المشكلة` لتسهيل الفهم على الفريق. ⓘ

كمسؤول الفريق Git

مسؤوليتك تشمل حماية الفروع الرئيسية، ومراجعة الكود، وإدارة عمليات الدمج.

مراجعة ودمج Pull Requests

```
# جلب التغييرات من عضو الفريق
git fetch origin feature/login

# راجع الكود محلياً
git checkout origin/feature/login

# ادمج بعد الموافقة
git checkout main
git merge --no-ff feature/login
git push origin main
```

حماية فرع main

```
# أنشئ فرع التطوير الرئيسي
git checkout -b develop

# ادفع الفرع إلى GitHub
git push origin develop

# ← ثم من GitHub: Settings > Branches
# أضف Branch protection rule على main
```

حل تعارضات الدمج

```
# عند ظهور تعارض بعد الدمج
git merge feature/login
# ← افتح الملف وحل التعارض يدوياً
# ← احذف علامات <<<====>>>>
# ثم أضف وأكمل
git add .
git commit -m "حل تعارض الدمج"
```

إدارة الإصدارات بال Tags

```
# أنشئ tag للإصدار
git tag -a v1.0.0 -m "الإصدار الأول"

# ادفع ال tag إلى GitHub
git push origin v1.0.0

# اعرض كل ال tags
git tag
```

رفع المشاريع إلى GitHub عبر Git

سواء كان مشروعاً جديداً أو موجوداً مسبقاً، هذه هي خطوات الربط والرفع.

↓ استنساخ مشروع من GitHub

```
# استنسخ مستودعاً موجوداً
git clone https://github.com/username/repo.git

# استنسخ في مجلد باسم محدد
git clone https://github.com/username/repo.git my-folder

# استنسخ فرعاً محدداً
git clone -b develop https://github.com/username/repo.git
```

🔑 الربط بـ SSH (للمحترفين)

```
# أنشئ مفتاح SSH
ssh-keygen -t ed25519 -C "email@example.com"

# اعرض المفتاح العام لنسخه إلى GitHub
cat ~/.ssh/id_ed25519.pub

# اختبر الاتصال
ssh -T git@github.com

# بدل SSH استخدم HTTPS
git remote add origin git@github.com:username/repo.git
```

✔️ ويُنصح به في بيئات العمل HTTPS أكثر أماناً وأسرع من SSH

☁ رفع مشروع جديد إلى GitHub

```
# 1. أنشئ مستودعاً جديداً على GitHub
# (من الموقع: New Repository)

# 2. اربط المستودع المحلي بـ GitHub
git remote add origin https://github.com/username/repo.git

# 3. أعد تسمية الفرع الرئيسي
git branch -M main

# 4. ارفع المشروع للمرة الأولى
git push -u origin main

# 5. للرفع في المرات القادمة
git push
```

🔗 ربط مشروع موجود بـ GitHub

```
# تحقق من الروابط الموجودة
git remote -v

# remote غير رابط الـ
git remote set-url origin https://github.com/username/new-repo.git

# احذف remote
git remote remove origin
```

إضافة الأعضاء ومنحهم الصلاحيات أو سحبها

تتم إدارة أعضاء الفريق من خلال إعدادات GitHub — لا تحتاج للطرفية في هذه الخطوة.



إدارة الفرق (Teams)

في GitHub Organizations يمكنك إنشاء فرق وتعيين صلاحيات جماعية:

أضف → Organization → Teams → New Team → الأعضاء وحدد الوصول للمستودعات



مستويات الصلاحيات

- **Read** — عرض الكود فقط
- **Triage** — بدون كتابة Issues إدارة
- **Write** — push وإنشاء فروع
- **Maintain** — إدارة المستودع
- **Admin** — صلاحيات كاملة



إضافة عضو

المسار: Repository → Settings → Collaborators → Add people

ابحث عن اسم المستخدم أو البريد الإلكتروني وأرسل الدعوة.



سحب الصلاحيات

المسار: Repository → Settings → Collaborators
ابحث عن العضو ثم اضغط **Remove** لإلغاء وصوله فوراً.



⚠ تأكد دائماً من منح أدنى مستوى من الصلاحيات اللازمة لكل عضو — لا تمنح Admin إلا لمن يحتاجه فعلاً.

التعامل مع GitHub & Git كعضو في الفريق

هذا هو سير العمل اليومي الذي يجب أن يتقنه كل عضو في الفريق لتجنب التعارضات وحماية الكود.

📋 سير العمل الكامل لعضو الفريق

```
# 1. استنسخ المشروع (مرة واحدة فقط).
git clone https://github.com/org/repo.git
cd repo
```

```
# 2. وحثه main تأكد أنك على
git checkout main
git pull origin main
```

```
# 3. أنشئ فرعاً خاصاً بمهمتك
git checkout -b feature/اسم-المهمة
```

```
# 4. اعمل وسجل تغييراتك
git add .
git commit -m "وصف واضح للتغيير"
```

```
# 5. ادفع فرعك إلى GitHub
git push origin feature/اسم-المهمة
```

```
# 6. من GitHub افتح Pull Request
# (Compare & pull request)
```

```
# 7. بعد الموافقة والدمج، نظّف
git checkout main
git pull origin main
git branch -d feature/اسم-المهمة
```

⚡ أوامر مهمة في بيئة الفريق

```
# جلب آخر تحديثات الفريق
git fetch origin
git pull origin main
```

```
# تحديث فرعك بآخر تغييرات
git rebase main
```

```
# حفظ عمل مؤقت دون
git stash
git stash pop # استرجعه لاحقاً
```

```
# اعرض من غيّر كل سطر
git blame filename.txt
```

```
# ابحث في سجل التغييرات
git log --oneline --graph --all
```

```
# محلياً تراجع عن آخر
git reset --soft HEAD~1
```

💡 **القاعدة الذهبية:** دائماً `git pull` قبل البدء بأي عمل جديد، وافتح Pull Request بدلاً من الدمج المباشر في main.

مرجع سريع — أهم أوامر Git

الإعداد

`git init` — تهيئة مستودع

`git clone [url]` — استنساخ

`git config --global` — الإعدادات

`git remote add origin` — ربط GitHub

الملفات وال Commits

`git status` — حالة الملفات

`git add .` — إضافة كل الملفات

`git commit -m ""` — تسجيل تغيير

`git log --oneline` — سجل مختصر

`git branch` — عرض الفروع

`git checkout -b [name]` — فرع جديد

`git merge [branch]` — دمج فرع

`git branch -d [name]` — حذف فرع

`git rebase main` — تحديث الفرع

`git stash` — حفظ مؤقت

`git stash pop` — استرجاع

الفروع

GitHub (Remote)

`git push origin main` — رفع

`git pull origin main` — سحب

`git fetch origin` — جلب بدون دمج

`git push -u origin main` — ربط وارفع

التراجع

`git reset --soft HEAD~1` — تراجع محلي

`git revert [hash]` — تراجع آمن

`git diff` — عرض التغييرات

الحفظ المؤقت



🎯 أنت الآن جاهز! ابدأ بإنشاء مستودعك الأول، واتبع الدليل خطوة بخطوة. الممارسة هي المفتاح — كل خطأ هو درس.